

Reinforcement Learning

LAB 4

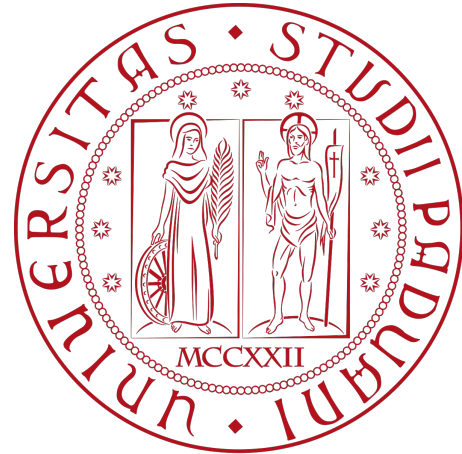
Monte Carlo prediction

Riccardo De Monte
riccardo.demonte@phd.unipd.it

Alberto Sinigaglia
alberto.sinigaglia@phd.unipd.it



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE



Once upon a time, in very dark times...

Last lab we saw prediction and control using dynamic programming, exploiting the knowledge of the state transition matrix $\mathbf{P}$ of the MDP. However, it might be that such transition is unknown, or too messy to deal with.

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Estimation of Value

However, recall that we did all that work in order to calculate the following (objectively horrible) quantity

$$V(s) := \sum_{s', r} p(s', r | s, \pi(s)) [r + \gamma V(s')]$$

Estimation of Value

However, recall that we did all that work in order to calculate the following (objectively horrible) quantity

$$V(s) := \sum_{s', r} p(s', r | s, \pi(s)) [r + \gamma V(s')]$$

But we saw that is equivalent to:

$$V(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \left[\sum_{s' \in \mathcal{S}} p(s' | s, a) \left(\gamma V(s') + \sum_{r \in \mathcal{R}} p(r | s, a, s') r \right) \right]$$

$$= \mathbb{E}_{a \sim \pi(a | s)} [\mathbb{E}_{s' \sim p(s' | s, a)} [\mathbb{E}_{r \sim p(r | s, a, s')} [r] + \gamma V(s')]]$$

$$= \mathbb{E}_{a \sim \pi(a | s)} [\mathbb{E}_{s' \sim p(s' | s, a)} [\mathbb{E}_{r \sim p(r | s, a, s')} [r + \gamma V(s')]]]$$

Estimation of Value

But this whole calculation was the Dynamic programming approach at calculating this (much simpler) quantity:

$$V(s) = \mathbb{E} \left[\sum_{i=t}^T \gamma^{i-t} r_{i+1} \mid s_t = s \right]$$

However, that expectation is not very well defined, meaning that if you want to write it down in close form, it includes (in a very hidden way) expectations over states, transitions, policy and so on (check the [3rd page of last lab notes](#)).

Estimation of Value

A (slightly) different notation used in RL literature:

given any s_t , $\tau = (s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, r_{t+2}, s_{t+2}, \dots, s_{T-1}, a_{T-1}, r_T, s_T)$

Estimation of Value

A (slightly) different notation used in RL literature:

given any s_t , $\tau = (s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, r_{t+2}, s_{t+2}, \dots, s_{T-1}, a_{T-1}, r_T, s_T)$

We can formally define the joint probability of any sequence of this kind. Since we are considering a MDP (markov property!),

$$P(\tau) = p(a_t|s_t)p(s_{t+1}, r_{t+1}|s_t, a_t)p(a_{t+1}|s_{t+1})p(s_{t+2}, r_{t+2}|s_{t+1}, a_{t+1}) \cdots p(s_T, r_T|s_{T-1}, a_{T-1})$$

Estimation of Value

A (slightly) different notation used in RL literature:

given any s_t , $\tau = (s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, r_{t+2}, s_{t+2}, \dots, s_{T-1}, a_{T-1}, r_T, s_T)$

We can formally define the joint probability of any sequence of this kind. Since we are considering a MDP (markov property!)

$$P(\tau) = p(a_t|s_t)p(s_{t+1}, r_{t+1}|s_t, a_t)p(a_{t+1}|s_{t+1})p(s_{t+2}, r_{t+2}|s_{t+1}, a_{t+1}) \cdots p(s_T, r_T|s_{T-1}, a_{T-1})$$

The action distribution is given by the policy:

$$P(\tau) = \pi(a_t|s_t)p(s_{t+1}, r_{t+1}|s_t, a_t)\pi(a_{t+1}|s_{t+1})p(s_{t+2}, r_{t+2}|s_{t+1}, a_{t+1}) \cdots p(s_T, r_T|s_{T-1}, a_{T-1})$$

Estimation of Value (actually, there is a mistake. Where?)

$$V(s) = \mathbb{E}_{\tau \sim P(\tau)} \left[\sum_{i=t}^T \gamma^{i-t} r_{i+1} \middle| s_t = s \right]$$

Estimation of Value

$$V(s) = \mathbb{E}_{\tau \sim P(\tau)} \left[\sum_{i=t}^T \gamma^{i-t} r_{i+1} \middle| s_t = s \right]$$

But maybe we can ignore the formalism... indeed this quantity is very simple to understand, it just mean “the average sum of reward starting from a specific state, and running a policy”

Engineers as soon as the
dimensionality of
a problem exceeds three.



Long live Monte Carlo

A quick recap on Monte Carlo:

“simulate it a bunch of times and average the result”

Long live Monte Carlo

In a more formal way, Monte Carlo is a way of estimating expectations, and is widely used all over applied mathematics (so make yourself comfortable with it)

It mainly comes from the central limit theorem, and it gives a cheap way of estimating expectations in an unbiased way:

$$\mathbb{E}[X] = \frac{\sum_{x_i \sim X} x_i}{N}$$

Long live Monte Carlo (slide borrowed from one of the last DRL lectures)

More generally we have that:

$$\mathbb{E}[x] \approx \frac{\sum_{x \sim p(x)} x}{N}, \quad \text{in general } \forall f \quad \mathbb{E}[f(x)] \approx \frac{\sum_{x \sim p(x)} f(x)}{N}$$

Long live Monte Carlo (slide borrowed from one of the last DRL lectures)

More generally we have that:

$$\mathbb{E}[x] \approx \frac{\sum_{x \sim p(x)} x}{N}, \quad \text{in general } \forall f \quad \mathbb{E}[f(x)] \approx \frac{\sum_{x \sim p(x)} f(x)}{N}$$

However, suppose to have any function $\mathbf{g}(\mathbf{x})$, that is not a PDF, and we want to integrate it, then we can reparametrize in the following way:

$$\int g(x) dx = \int \frac{p(x)}{p(x)} g(x) dx = \int \frac{g(x)}{p(x)} p(x) dx \approx \frac{\sum_{x \sim p(x)} g(x)/p(x)}{N}$$

Thus, we can **sample values from a known PDF p , evaluate g on that point, and sum all the resulting values**, and we are guaranteed to converge (at some point)

Long live Monte Carlo

This should also sound like off policy learning.

$$\int g(x) dx = \int \frac{p(x)}{p(x)} g(x) dx = \int \frac{g(x)}{p(x)} p(x) dx \approx \frac{\sum_{x \sim p(x)} g(x)/p(x)}{N}$$

Indeed, off-policy is the “art of learning from somebody else experience”.. but:

$$\begin{aligned} \mathbb{E}_{x \sim p(x)} [f(x)] &\approx \frac{\sum_{i=0}^N f(x_i)}{N} \quad x_i \sim p(x) = \sum_{x \in \mathcal{X}} p(x) f(x) \\ &= \sum_{x \in \mathcal{X}} \frac{q(x)}{q(x)} p(x) f(x) = \sum_{x \in \mathcal{X}} \frac{p(x)}{q(x)} q(x) f(x) \\ &= \sum_{x \in \mathcal{X}} q(x) \frac{p(x)}{q(x)} f(x) = \mathbb{E}_{x \sim q(x)} \left[\frac{p(x)}{q(x)} f(x) \right] \\ &\approx \frac{\sum_{i=0}^N \frac{p(x_i)}{q(x_i)} f(x_i)}{N} \quad x_i \sim q(x) \end{aligned}$$

Long live Monte Carlo

Specifically, in our case, we are interested in a specific expectation:

$$\mathbb{E}[X] = \frac{\sum_{x_i \sim X} x_i}{N} \Rightarrow V(S) = \mathbb{E}[G_t | S_t = \mathbf{s}]$$

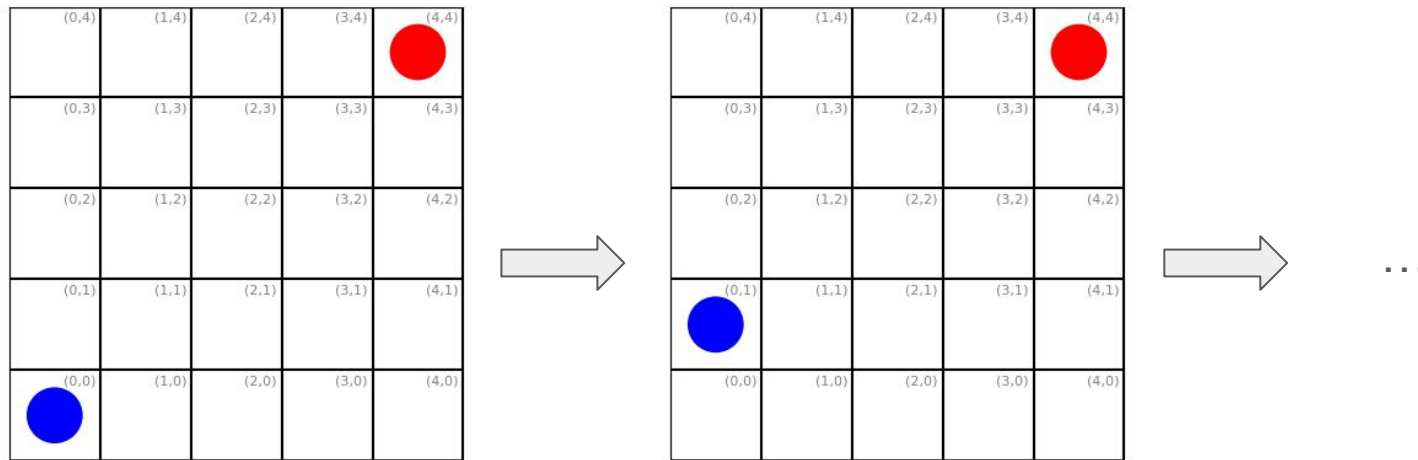
Monte Carlo for prediction

At this point, we have a feasible way of approximating that quantity, which can be summarized as:

*“let’s put the agent in a specific state, run the policy for a while and for a bunch of times,
and average the results”*

Environment

For this lab, we will use a new environment, which is a simple gridworld with some “wind”. Since we are doing prediction, we get to pick the policy, and we are only asked to estimate its value function. The blue ball is the player, red the goal.



Value function

For sake of simplicity, we will use as 2D array as value function (to track x/y coordinates of the player) and for simplicity, we will keep the initial point and goal on the same location. As seen during lecture, you have 2 choices for estimating the values.

First visit

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Every visit

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

~~Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:~~

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Competition: develop the best policy

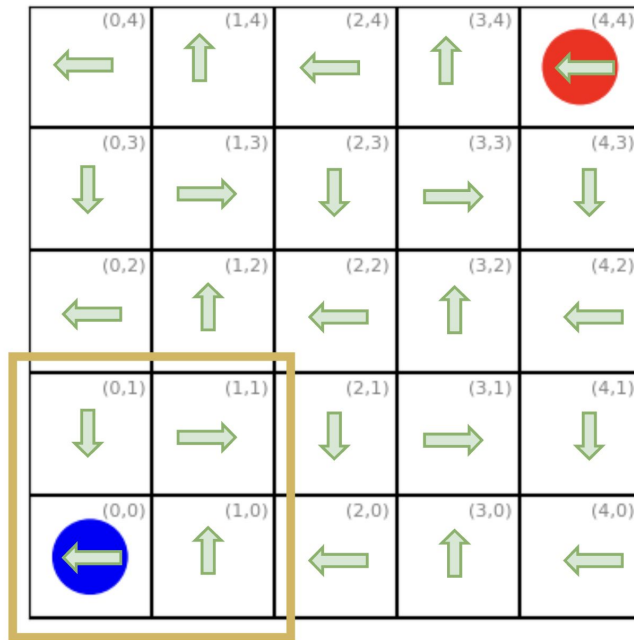
Now the environment is highly stochastic.

At any position, there is either some wind in the X direction or Y direction.

Particularly, the wind is either from $[-2, -1, 1]$ or $[-1, 1, 2]$.

In the cells where the arrow are horizontal, there is wind in the X direction, opposite for vertical ones.

The arrow point in the direction of max wind.



But how do understand how good is your policy?